

Mitra Query Hub V2 - Makeover Plan

Prepared for: Vivek M + Intern

Date: 2026-08-13

1. Purpose & Scope

This plan covers the V2 makeover of `query_hub` requested to support the Mitra Conversational Insights POC. It is scoped to five concrete changes:

1. Proper project structure (currently five flat `.py` files with no package layout, no tests, no docs).
2. A dedicated application database to store user queries, AI responses, and feedback (currently nothing is persisted — see prior review of `utils.py`).
3. Switch the LLM provider from direct OpenAI to OpenRouter, so models can be swapped without code changes.
4. Split the Mitra system prompt (currently one large block in `prompts.py`) into stage-tagged pieces stored in the new database, fetched per-stage at runtime instead of sent as one static prompt.
5. All of the above lands on a new branch, separate from `main`.

This document does **not** cover deterministic Text-to-SQL retrieval or the Sheets→DB real-time sync described in the original requirements note's "Build Plan" section — those remain out of scope until this makeover is done, per earlier agreement.

2. Current State (baseline)

- **Entry point:** `app.py` — Streamlit UI, one sidebar radio to pick a dataset (`MENTORING` , `SCP` , `PROJECTS` , `KATHA`).
- **Chain logic:** `utils.py` — `create_sql_query_chain` (LangChain) generates SQL against Postgres/MySQL, executes it, rephrases the result via `answer_prompt` .
- **Prompts:** `prompts.py` — one generic `answer_prompt` , plus the new `mitra_system_prompt` / `mitra_answer_prompt` added during the prompt-review pass (not yet wired into a chain).

- **No persistence** of queries, responses, or feedback. Streamlit `session_state` holds chat history in-memory only, lost on refresh, and isn't actually consumed by the SQL-generation prompt today (dead code).
- **No Mitra DB connection yet** — confirmed to already exist as a database (not raw Sheets), but connection details/table descriptions are still pending from Vivek.
- **LLM provider:** `ChatOpenAI(model="gpt-3.5-turbo")` called directly against OpenAI.

3. Target Architecture

3.1 Project Structure

```

query_hub/
├── app/
│   ├── main.py                # Streamlit entrypoint (was app.py)
│   └── components/
│       ├── sidebar.py         # dataset selector
│       ├── chat.py            # message rendering
│       └── feedback_widget.py  # thumbs up/down + comment box
├── core/
│   ├── llm/
│   │   └── client.py           # OpenRouter client factory (model/provider config-driven)
│   ├── chains/
│   │   ├── retrieval.py       # create_sql_query_chain wiring per dataset (existing logic, relocated)
│   │   └── interpretation.py   # prompt assembly + answer generation
│   └── context/
│       └── history.py          # session-backed message history (replaces Streamlit-only state)
├── db/
│   ├── models.py              # SQLAlchemy models: Session, Query, Feedback, Prompt, ModelConfig
│   ├── engine.py              # engine/session factory (SQLite by default)
│   ├── migrations/            # Alembic migrations
│   └── repositories/
│       ├── query_repo.py
│       ├── feedback_repo.py
│       ├── prompt_repo.py
│       └── session_repo.py
├── data/
│   ├── table_desc_mentoring.csv # existing table descriptions, relocated as-is
│   ├── table_desc_scp.csv
│   ├── table_desc_projects.csv
│   ├── table_desc_katha.csv
│   ├── table_desc_mitra.csv     # new, once MITra schema is available
│   └── seed_prompts.py          # one-time script: loads mitra_system_prompt sections into db.prompts
├── config/
│   └── settings.py             # env loading: DB creds per dataset, OpenRouter key, app DB path
└── tests/
    ├── test_db_models.py
    ├── test_repositories.py
    ├── test_prompt_assembly.py
    └── test_llm_client.py

```

```

├─ docs/
│   └─ planning/v2-makeover/      # this plan
├─ requirements.txt
└─ README.md

```

Existing dataset logic (`MENTORING / SCP / PROJECTS / KATHA`) is relocated, not rewritten — the makeover changes *how* things are wired (structure, persistence, provider, prompt source), not the retrieval approach itself.

3.2 Application Database

A new, dedicated SQLite database (`query_hub.db`) separate from the four analytical databases being queried. SQLite is the right default here — zero new infra to stand up, works fully through SQLAlchemy, and migrating to Postgres later (if usage grows past POC scale) is a connection-string change, not a rewrite. See Section 5 for schema.

This directly satisfies requirement #2 and the original requirements doc's Query Capture section (query id, session id, timestamp, query, response, feedback, dataset) and User Feedback section (thumbs up/down + optional comment).

3.3 LLM Provider: OpenRouter

Replace direct `ChatOpenAI(model="gpt-3.5-turbo")` calls with an OpenRouter-backed client.

OpenRouter is OpenAI-API-compatible, so this is a `base_url` + `api_key` + model-name change, not a new SDK:

```

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
    model=active_model_config.model_name,    # e.g. "openai/gpt-4o-mini", "anthropic/claude-sonnet-5"
    temperature=0,
    default_headers={"HTTP-Referer": "query-hub-poc", "X-Title": "MItra Query Hub"},
)

```

Model choice becomes a config row (see `model_config` table, Section 5) rather than a hardcoded string — swapping models for a test is a data change, not a deploy.

3.4 Prompt Store & Stage-Based Assembly

Per requirement #5: instead of sending `mitra_system_prompt` as one large static block, its twelve sections are split into stage-tagged rows in a `prompts` table and reassembled per request.

Proposed stage split (from the already-drafted content in `prompts.py`):

Stage key	Source section(s)	Always included?
<code>scope_guard</code>	Data Sources - Single Source of Truth; Filters Stay Locked	Yes
<code>response_length</code>	Response Length: Match the Question, Don't Over-Answer	Yes
<code>chart_policy</code>	Charts - Only When Explicitly Requested	Only if chart-related keywords detected
<code>citation</code>	Cite Your Sources	Yes
<code>tone_rules</code>	Avoid Sweeping Conclusions; Plain Language, No Jargon	Yes
<code>percentage_format</code>	Percentages Always Need Base Numbers	Yes
<code>executive_structure</code>	Executive Insight Format (13-part format)	Only when a detailed/exploratory answer is triggered
<code>language_handling</code>	Language Handling	Yes
<code>privacy</code>	Privacy	Yes
<code>data_quality</code>	Data Quality (theme reference list)	Only when the query touches themes

A `PromptAssembler` (`core/chains/interpretation.py`) fetches the active row for each applicable stage key, concatenates them into the final system prompt, and passes it to the OpenRouter client. This means a wording fix to one rule (e.g., the percentage-format rule) is a DB update, not a code change — and stages that don't apply to a given question (e.g., `chart_policy` on a query with no chart request) don't inflate the prompt.

3.5 Context Retention

Continues the prompt-based approach already agreed on: `MessagesPlaceholder` fed with real conversation history. The one structural change from today is that history is read from the new `sessions / queries` tables (survives a page refresh) instead of only `st.session_state` (lost on refresh). Slot-based context tracking (state/district/program/etc.) remains a possible V3 upgrade, not in this makeover.

3.6 Mltra Dataset Integration

Added as a fifth dataset option alongside the existing four, using the same `create_sql_query_chain` retrieval pattern, but routed to `mitra_answer_prompt / PromptAssembler` for interpretation instead of the generic `answer_prompt`. Blocked on: Mltra DB connection details + table descriptions (open item, owned by Vivek — see Section 9).

4. End-to-End Flows

4.1 Chat request flow

1. User picks a dataset and asks a question (`app/main.py`).
2. Session resolved or created (`session_repo`) — a stable `session_id` persists across the browser session.
3. Prior turns loaded from `queries` table for that session, fed into `MessagesPlaceholder`.
4. Retrieval chain (`core/chains/retrieval.py`) generates and executes SQL against the selected dataset's DB.
5. For Mltra: `PromptAssembler` fetches applicable stage rows from `prompts`, builds the system prompt.
6. OpenRouter client (`core/llm/client.py`) generates the final answer.
7. Response rendered in the UI.
8. Query + response logged to `queries` (query id, session id, timestamp, question, generated SQL, answer, dataset).
9. Feedback widget shown; a 👍/👎 (+ optional comment) writes to `feedback`, keyed by `query_id`.

4.2 Prompt management flow

1. `data/seed_prompts.py` runs once to load the twelve sections into `prompts`, tagged by stage, versioned.
2. To change wording: insert a new version row and flip `is_active`, or edit in place for the POC — either way, no redeploy.

4.3 Model-swap flow

1. `model_config` holds one active row per purpose (`retrieval` vs `interpretation`).
2. To try a different model: flip `is_active` to a different config row. `core/llm/client.py` reads the active row at call time.

5. Database Schema (`query_hub.db`)

```

sessions
  session_id      TEXT PRIMARY KEY
  dataset_option  TEXT NOT NULL      -- MENTORING | SCP | PROJECTS | KATHA | MITRA
  created_at      TIMESTAMP NOT NULL

queries
  query_id        TEXT PRIMARY KEY
  session_id      TEXT NOT NULL REFERENCES sessions(session_id)
  timestamp       TIMESTAMP NOT NULL
  user_query      TEXT NOT NULL
  generated_sql    TEXT
  ai_response     TEXT NOT NULL
  dataset_option  TEXT NOT NULL
  relevant_tables TEXT              -- optional, comma-separated

feedback
  feedback_id     TEXT PRIMARY KEY
  query_id        TEXT NOT NULL REFERENCES queries(query_id)
  rating          TEXT NOT NULL      -- 'up' | 'down'
  comment         TEXT
  created_at      TIMESTAMP NOT NULL

prompts
  prompt_id       TEXT PRIMARY KEY
  stage_key       TEXT NOT NULL      -- e.g. 'response_length'
  name            TEXT NOT NULL
  content         TEXT NOT NULL
  version         INTEGER NOT NULL
  is_active       BOOLEAN NOT NULL DEFAULT TRUE
  updated_at      TIMESTAMP NOT NULL

model_config
  config_id       TEXT PRIMARY KEY
  purpose         TEXT NOT NULL      -- 'retrieval' | 'interpretation'
  provider        TEXT NOT NULL      -- 'openrouter'
  model_name      TEXT NOT NULL      -- e.g. 'anthropic/claude-sonnet-5'
  temperature     REAL NOT NULL DEFAULT 0
  is_active       BOOLEAN NOT NULL DEFAULT TRUE

```

6. Git & Branching Plan

- New branch off `main`: `v2-makeover` (naming suggestion — confirm before creating).
- All five changes land here; `main` stays untouched and deployable until this is reviewed and merged.
- Suggested convention: intern commits to `v2-makeover` directly or to short-lived sub-branches (`v2-makeover-db`, `v2-makeover-structure`) merged back into `v2-makeover` frequently, to avoid a single giant diff at the end of 15 days.
- No branch has been created yet — this happens once the plan below is confirmed.

7. 15-Day Delivery Plan

Two tracks run in parallel with minimal cross-blocking. The intern's track is self-contained new infrastructure (project structure, DB layer, prompt seed data, feedback UI, repositories, tests, docs) — nothing on it requires Vivek's track to be finished first. Vivek's track owns the pieces that need external credentials or architecture-sensitive judgment calls (OpenRouter setup, Mltra connection, prompt assembler, context retention, final integration). The two tracks meet at integration (Days 11–13).

Phase 1 — Foundation (Days 1–3) Phase 2 — Core Build (Days 4–10) Phase 3 — Integration & Handoff (Days 11–15)

Full day-by-day task list with owners is in `task_assignments.csv` (companion file). Summary by phase:

- **Days 1–3:** New project structure and file relocation (Intern); branch creation + OpenRouter account/key setup (Vivek, in parallel).
- **Days 2–5:** DB models + migrations for all five tables (Intern); LLM client factory against OpenRouter (Vivek).
- **Days 4–7:** Prompt seed script splitting `mitra_system_prompt` by stage (Intern); Mltra DB connection + table descriptions (Vivek, pending info Vivek already owns).
- **Days 6–9:** Feedback UI widget + repositories (Intern); Prompt Assembler component (Vivek).
- **Days 8–10:** Query/response logging wired into the chat flow (Intern); session-backed context retention (Vivek).
- **Days 10–12:** Unit tests + docs (Intern); integration of both tracks into one working chain (Vivek).
- **Days 12–15:** End-to-end testing across all five datasets and model swaps, buffer for fixes, PR opened against `main` (Vivek + Intern together).

8. Open Dependencies / Risks

- **Mitra DB connection details** (host/credentials, engine type, table descriptions) — still pending from Vivek; blocks only the Mitra-specific dataset option, not the rest of the makeover.
- **OpenRouter account & billing** — needs to be set up before Day 3; without it, the LLM client factory can be built and unit-tested against a mock but not run live.
- **Theme taxonomy in the Mitra DB** — confirmed themes are already a DB column; if the actual values don't match the 10-item reference list drafted in `mitra_system_prompt`'s Data Quality section, that section needs a content update (data change, not a structural one, given the new prompt store).
- **Intern ramp-up time** — 15 days is tight for someone new to the codebase; Phase 1 is deliberately low-ambiguity (scaffolding, schema-from-spec) to get early wins before anything judgment-heavy.

9. Definition of Done

- `v2-makeover` branch contains: new project structure, working SQLite-backed query/feedback/prompt store, OpenRouter-driven LLM calls for all five dataset options, Mitra prompt served from the stage-based store, and feedback capture visible in the UI.
- All existing four datasets (MENTORING/SCP/PROJECTS/KATHA) still work end-to-end after the restructure.
- A query asked twice in the same session returns consistent context handling (prompt-based retention verified manually).
- README documents setup (env vars, DB init, running the app) well enough that a new contributor doesn't need this document to get started.